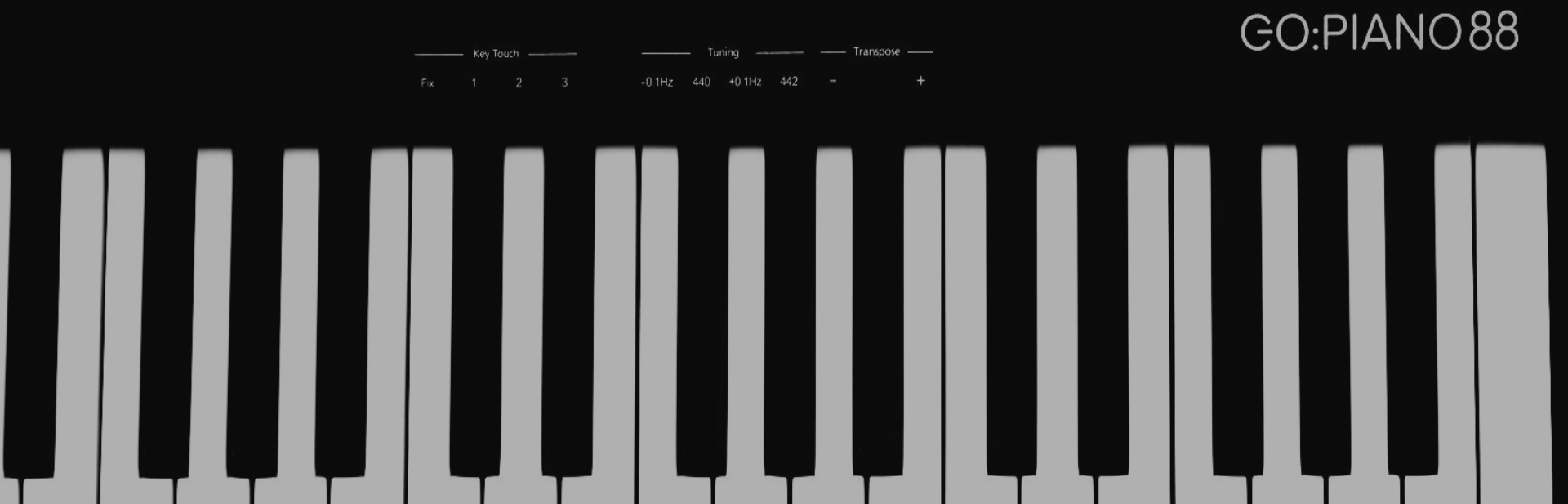
MIDI Performance Signature Behavioural Biometric Identification

Blain Polishak - T00723036 Aidan Evans - T00708916



Research Question

Can a behavioural biometric system accurately identify individual musicians, when compared against a dataset of known musicians, by analyzing motor, rhythm, and stylistic behavioural signatures collected via the MIDI data of a single keyboard performance?

OBJECTIVE

- Develop a system which can identify individual musicians using MIDI performance data
- Capture basic performance metrics such as timing, release velocity, pause-duration, etc.
- Construct compound metrics such as rhythm, range of pitch, average arpeggiation, polyphony, etc., from the raw performance metrics
- Apply machine learning models to learn each musician's distinct behavioural styles and identify musicians from new performance data.

HYPOTHESIS

- Every musician has a distinct “behavioural signature”, rooted in their individual motor habits.
- These micro-signatures can be captured in MIDI performance data and analyzed.
- Machine learning models are advanced enough to identify performers by these behavioural signatures, by analyzing the MIDI data of a new performances given previous training data
- This system will achieve an identification accuracy significantly higher than random probability, on a limited dataset, although a more robust and detailed dataset would greatly improve this accuracy.

LITERATURE REVIEW

- Behavioural consistency gives traits that remain identifiable even when performers play different pieces [1,2,3]
- Micro-variations in timing (inter-onset intervals), velocity levels, and note articulation serve as reliable fingerprints in biometric data [3,4]
- Deep learning models have proven that a computer can identify performers by analyzing layered patterns in their performances [2,5]
- The PiJAMA study [6] demonstrates that these models can work and be accurate at scale
- Finger Kinematics research shows that a pianists “key velocity” and “timing” patterns are physically ingrained motor habits which remain adequately consistent [1,7]

JUSTIFICATION OF STUDY

- Piano keyboard biometrics has not been studied using freeform jazz improvisation.
- Existing biometric systems often **require specialized hardware** (e.g., cameras, fingerprint sensors), limiting accessibility
- Many behavioral biometric methods remain **research-focused and not practical** for real-world deployment
- Current systems prioritize **accuracy over usability**
- Most approaches **rely on controlled lab environments**, not real-world conditions, reducing practical applicability
- There is a **need for a low-cost, easily deployable biometric system** that uses widely available tools like MIDI

INITIAL DATASET

- Note data from each performance contains only 9 columns, but thousands of rows

event_id	time_ms	tick	note	note_name	velocity_on	time_off_ms	tick_off	velocity_off	duration_ms
0	0	135	64	E4	55	196	179	36	196
1	125	163	66	F#4	61	335	210	23	210
2	879	332	64	E4	57	1237	412	36	357
3	1196	403	66	F#4	57	1326	432	54	129

- A python script performs feature extraction which combines and averages behaviours to create compound features like average pitch and average polyphony (simultaneous notes)

```
# ----------------------------------------------------------------------
def compute_ioi(times_ms):
    times = np.sort(times_ms)
    if len(times) < 2:
        return np.array([np.nan])
    return np.diff(times)

def compute_articulation(durations_ms, ioi_values):
    n = min(len(durations_ms), len(ioi_values))
    if n == 0:
        return np.nan

    ratios = np.array(durations_ms[:n]) / np.where(ioi_values[:n] == 0, np.nan, ioi_values[:n])
    return np.nanmean(ratios)
```

```
# ----------------------------------------------------------------------
# Polyphony
# ----------------------------------------------------------------------
def compute_polyphony(df_win):
    active_counts = []
    times = df_win["time_ms"].values

    for t in times:
        active = np.sum(
            (df_win["time_ms"] <= t) &
            (df_win["time_off_ms"] > t)
        )
        active_counts.append(active)

    return np.mean(active_counts)
```

COMPOUND DATASET FEATURES LIST

- **avg_velocity**: Mean note-on velocity (loudness) within the window.
- **avg_velocity_off**: Mean note-off velocity (release intensity) within the window.
- **avg_ioi**: Average inter-onset interval between consecutive notes.
- **avg_duration**: Mean duration of notes in milliseconds.
- **avg_articulation**: Average ratio of note duration to IOI, indicating legato vs staccato.
- **pitch_range**: Difference between highest and lowest pitch in the window.
- **avg_pitch**: Mean pitch value of all notes.
- **avg_pitch_step**: Average absolute pitch change between consecutive notes.
- **num_chords**: Number of detected arpeggio-like chord sequences.
- **avg_notes_per_chord**: Mean number of notes per detected chord/arpeggio.
- **chord_density**: Number of chords per second within the window.
- **avg_polyphony**: Average number of simultaneously sounding notes.
- **avg_arpeggiation_speed**: Mean time between successive notes in arpeggios.
- **avg_arpeggation_distance**: Mean pitch interval between successive arpeggio notes.
- **num_grace_notes**: Count of detected grace notes.
- **avg_grace_duration**: Mean duration of grace notes.
- **avg_grace_interval**: Mean pitch interval between grace notes and their following notes.

COMPOUND DATASET

- Once all features are available, each individual performance is combined into a stacked training dataset with userIDs used to track users
- All non-entry features are zeroed in this step,
  and no further feature engineering is necessary for training
  (outside of ignoring window_id, time, and userID.)

	userID	window_id	time_ms	avg_velocity	avg_velocity_off	avg_ioi	avg_duration	avg_articulation	pitch_range	avg_pitch	avg_pitch_step	num_chords	avg_notes_
0	0	0	0	57.164	37.344	249.633	468.295	15.257	20	62.000	4.100	4	
1	0	1	15000	58.329	34.500	181.938	570.524	28.903	22	62.037	5.975	10	
2	0	2	30000	57.968	37.387	223.721	593.629	21.932	20	62.403	5.016	4	
3	0	3	45000	60.000	39.443	190.256	501.089	21.050	33	60.899	5.923	8	
4	0	4	60000	64.534	35.753	200.514	543.644	17.184	24	62.014	5.431	8	

SOURCE COLLECTION

- **Collection methodology:** Data was collected on a consumer grade M-Audio MIDI keyboard via FL Studio and/or a Python script.
- **Musicians in the training dataset:** Aidan Evans, Renz Tingson, Eric Kitt, Kei Massalski, Marvellous Ifezue
- **Song used:** Jazz standard *Autumn Leaves* in E minor
- Musicians were prompted to play with a laid-back mood, around 90bpm

MODELS

- Three compound models were utilized:
  - **Model 1:** *Stability Voting* using the results of Extra Trees, Logistic Regression, and KNN
  - **Model 2:** *De-Noising Pipeline* which uses StandardScaler, PCA, and Logistic Regression
  - **Model 3:** *Conservative Model* which uses Random Forest and KNN to make predictions before a Logistic Regression model decides the best conclusion
- Each model utilized a 80-20 training/testing split

MODEL 1: STABILITY VOTING

- Logistic Regression, KNN, and Extra Trees all come to their own independent conclusions on the result of the data.
- Each model then evaluates its own confidence in the results it gave and comes up with a confidence score.
- All of the models' results are then combined with their average confidence scores and averaged out to get a final result which has the most confidence.

MODEL 2: DE-NOISING

- StandardScaler (Leveler) normalizes the importance of the data and ensures that outliers don’t drown out other features
- PCA (Filter) removes random noise and erroneous results, like one-off errors.
- Logistic Regression (Decision Maker) decides which user’s style most accurately matches that shown in the leveled and filtered data

MODEL 3: CONSERVATIVE APPROACH

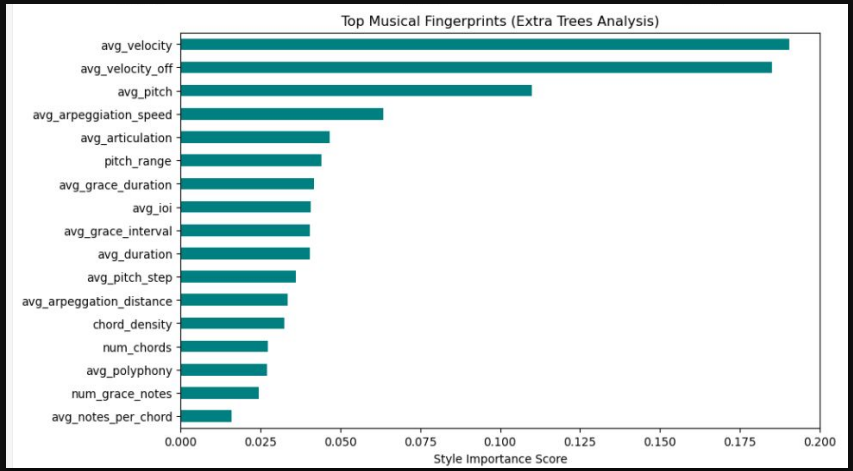
- Random Forest and KNN find complex behavioural patterns in the data
- Each model reports its data and its “confidence” in its findings to Logistic Regression, which acts as a manager
- Logistic Regression uses a simple linear formula to weigh these opinions, choosing the most robust and repeatable option to prevent low-occurrence but highly influential patterns from incorrectly influencing decisions

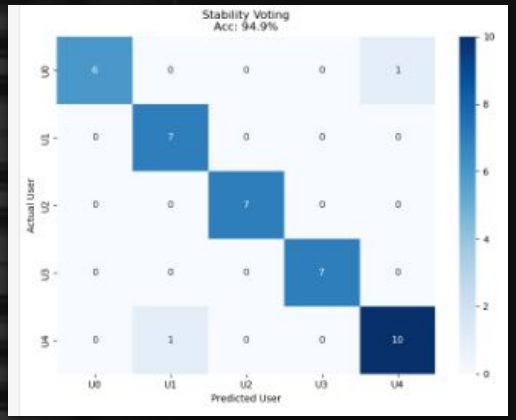
RESULTS : TRAINING

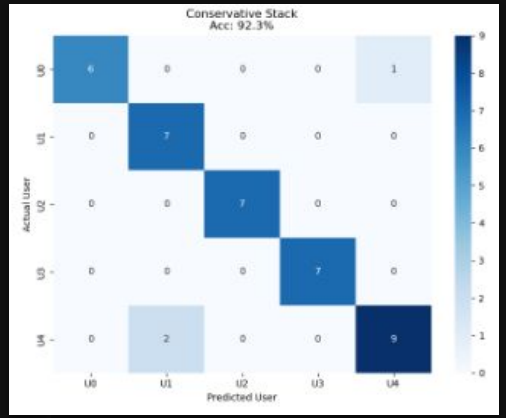
```
--- Training Stability Voting ---
Stability Voting Accuracy: 94.87%
              precision    recall  f1-score   support

           0       1.00      0.86      0.92         7
           1       0.88      1.00      0.93         7
           2       1.00      1.00      1.00         7
           3       1.00      1.00      1.00         7
           4       0.91      0.91      0.91        11

    accuracy                           0.95        39
   macro avg       0.96      0.95      0.95        39
weighted avg       0.95      0.95      0.95        39

--- Training De-Noising (PCA+LR) ---
De-Noising (PCA+LR) Accuracy: 94.87%
              precision    recall  f1-score   support

           0       1.00      0.86      0.92         7
           1       0.88      1.00      0.93         7
           2       1.00      1.00      1.00         7
           3       1.00      1.00      1.00         7
           4       0.91      0.91      0.91        11

    accuracy                           0.95        39
   macro avg       0.96      0.95      0.95        39
weighted avg       0.95      0.95      0.95        39

--- Training Conservative Stack ---
Conservative Stack Accuracy: 92.31%
              precision    recall  f1-score   support

           0       1.00      0.86      0.92         7
           1       0.78      1.00      0.88         7
           2       1.00      1.00      1.00         7
           3       1.00      1.00      1.00         7
           4       0.90      0.82      0.86        11

    accuracy                           0.92        39
   macro avg       0.94      0.94      0.93        39
weighted avg       0.93      0.92      0.92        39
```

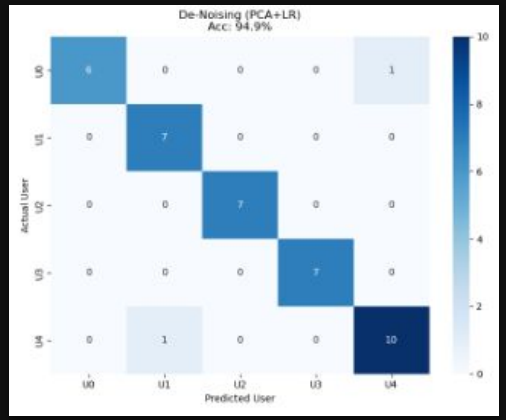
```
--- Final Performance Comparison ---
                  Model  Accuracy     Trustworthiness
0     Stability Voting  0.948718      High (Diverse)
1  De-Noising (PCA+LR)  0.948718  Highest (Filtered)
2   Conservative Stack  0.923077  Moderate (Complex)
```

RESULTS : TRAINING









RESULTS : TEST DATA

```
Analyzing Recording: AidanTest.csv
Windows detected: 13
------------------------------
                Model Predicted User Confidence Score
0     Stability Voting         User 4            81.02%
1  De-Noising (PCA+LR)         User 4            79.99%
2   Conservative Stack         User 4            84.42%
```

```
# Use the filename of the CSV you want to test
report = predict_new_recording('AidanOldTest3.csv')
print(report)
```

```
Analyzing Recording: AidanOldTest3.csv
Windows detected: 15
------------------------------
                Model Predicted User Confidence Score
0     Stability Voting         User 4            67.47%
1  De-Noising (PCA+LR)         User 4            72.25%
2   Conservative Stack         User 4            70.86%
```

```
Analyzing Recording: AidanOldTest.csv
Windows detected: 8
------------------------------
                Model Predicted User Confidence Score
0     Stability Voting         User 4            69.59%
1  De-Noising (PCA+LR)         User 4            86.06%
2   Conservative Stack         User 4            70.96%
```

```
        inference_results.append({
            "Model": name,
            "Predicted User": f"User {predicted_user}",
            "Confidence Score": f"{confidence:.2f}%"
        })

    # Display as a clean summary table
    return pd.DataFrame(inference_results)

# --- EXECUTION ---
# Use the filename of the CSV you want to test
report = predict_new_recording('filled_features1.csv')
print(report)
```

```
Analyzing Recording: filled_features1.csv
Windows detected: 35
------------------------------
                Model Predicted User Confidence Score
0     Stability Voting         User 1            87.52%
1  De-Noising (PCA+LR)         User 1            77.62%
2   Conservative Stack         User 1            86.99%
```

FUTURE IMPROVEMENTS

- More features
- Larger dataset with many performances
- “Aftertouch” MIDI keyboard for pressure sensitivity monitoring
- Integration into an automatic continuous authentication application
- Dataset using multiple different standards and BPMs instead of the same standard for all performances
- Test more variables (window size, different averages, etc)
  - Unable to test properly without more data, because accuracy is already high

REFERENCES

1. E. Stamatatos and G. Widmer, “Automatic identification of music performers with learning ensembles,” *Artificial Intelligence*, vol. 165, no. 1, pp. 37–56, 2005.
2. J. Tang, G. Wiggins, and G. Fazekas, “Pianist Identification Using Convolutional Neural Networks,” *arXiv.org*, 2023.
3. H. Cheston, J. L. Schlichting, I. Cross, and P. M. C. Harrison, “Rhythmic Qualities of Jazz Improvisation Predict Performer Identity and Style in Source-Separated Audio Recordings,” *Royal Society Open Science*, vol. 11, no. 11, 2024.
4. R. Syed, G. Fazekas, and G. A. Wiggins, “Performer Identification From Symbolic Representation of Music Using Statistical Models,” *arXiv.org*, 2021.
5. J. Yang, Y. Zhou, and Y. Lu, “Multimedia Identification and Analysis Algorithm of Piano Performance Music Based on Deep Learning,” *J. Electrical Systems*, vol. 19, no. 4, pp. 196–210, 2023.
6. D. Edwards, S. Dixon, and E. Benetos, “PiJAMA: Piano Jazz with Automatic MIDI Annotations,” *Transactions of the International Society for Music Information Retrieval*, vol. 6, no. 1, pp. 89–102, 2023.
7. S. Dalla Bella and C. Palmer, “Rate Effects on Timing, Key Velocity, and Finger Kinematics in Piano Performance,” *PLoS ONE*, vol. 6, no. 6, p. e20518, 2011.